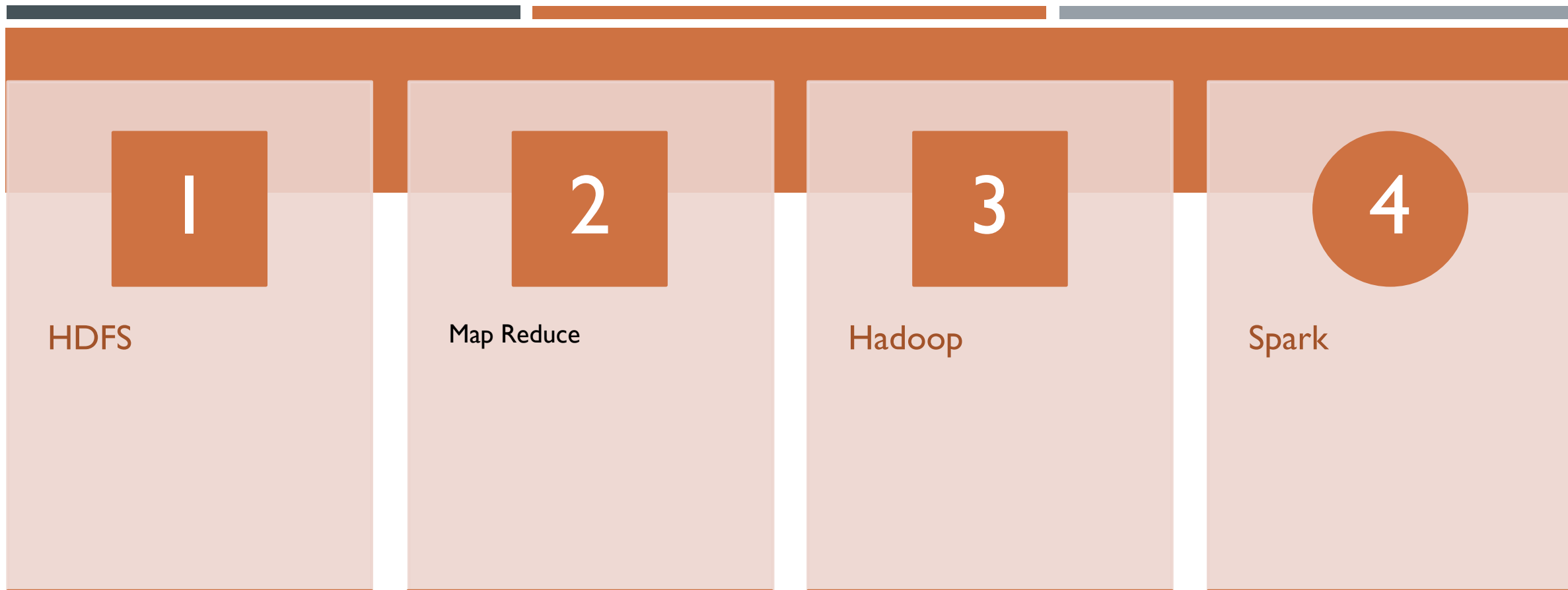




ADVANCED DATA MANAGEMENT

when the going gets big, bigs get going

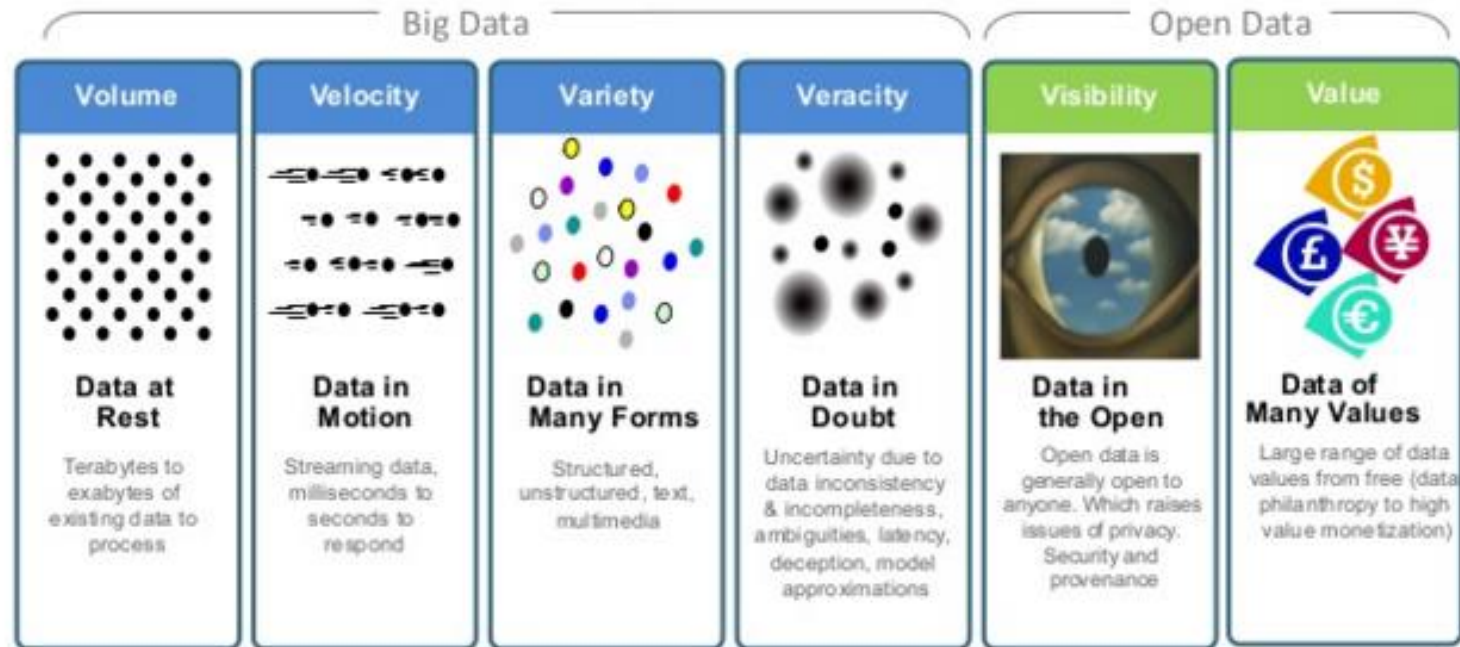


BIG DATA

Big Data & Analytics



Big Data is not 'just' data, there are a few new considerations



BIG DATA

- Analyze big data on a single «big» server is
 - slow
 - Costly
 - Hard to realize
- The solution
 - Distribute the analysis over a set of inexpensive hardware commodities → parallel system
 - Main issues
 - Synchronization
 - Deadlock,
 - Bandwidth
 - Coordination
 - Failure

BIG DATA ARCHITECTURE: BASIC FEATURES

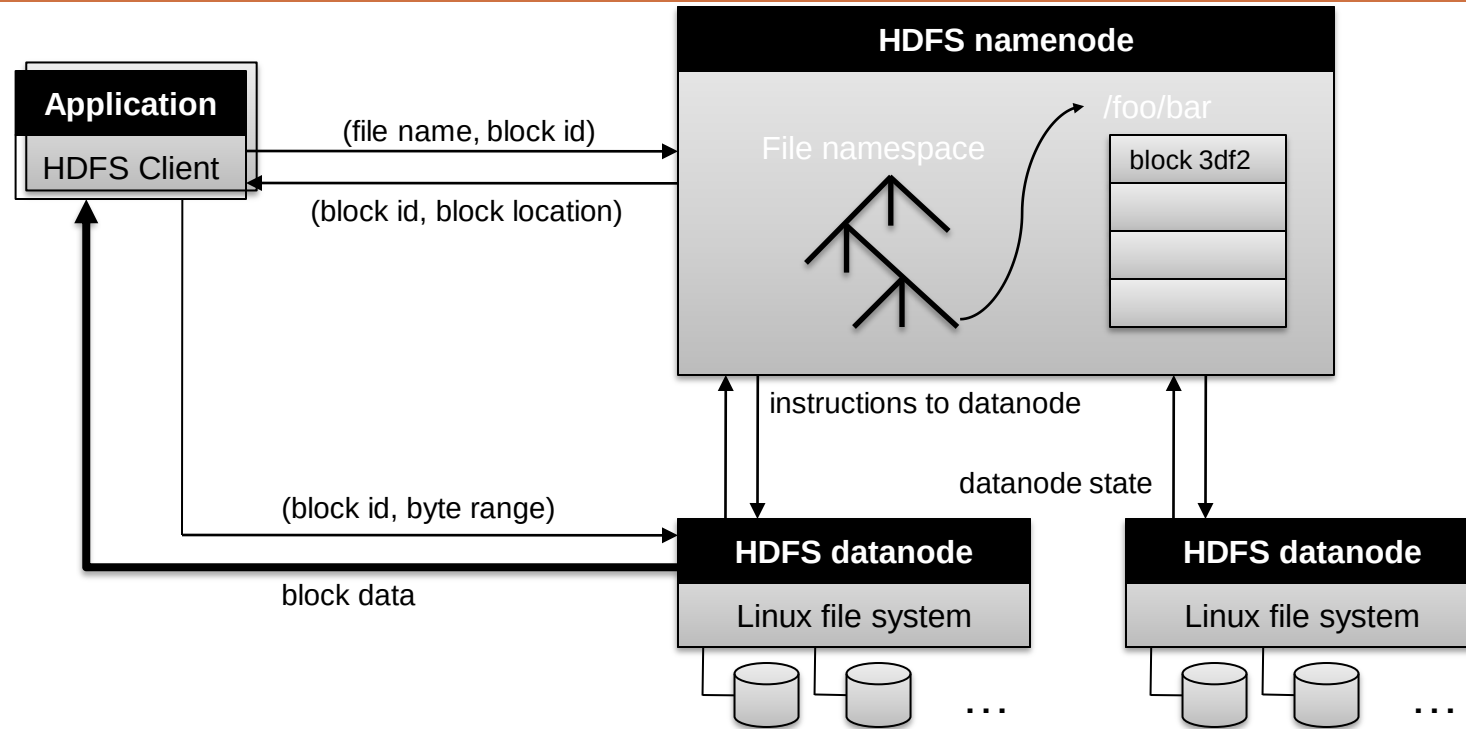
- Linear scalability (scale out)
- Computation task moves towards data and not vice-versa
- Failure management
- Works on commodity hardware



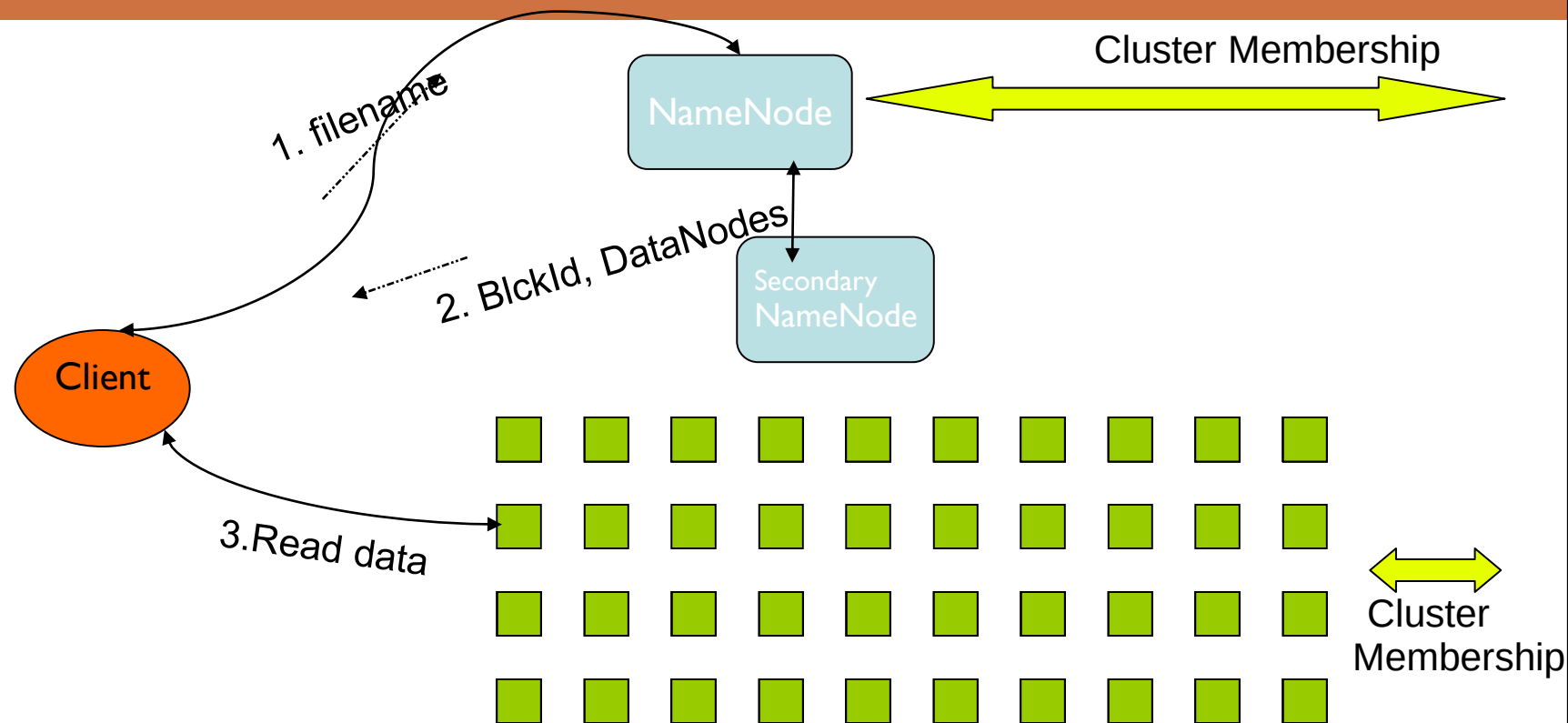
HDFS



HDFS



HDFS ARCHITECTURE



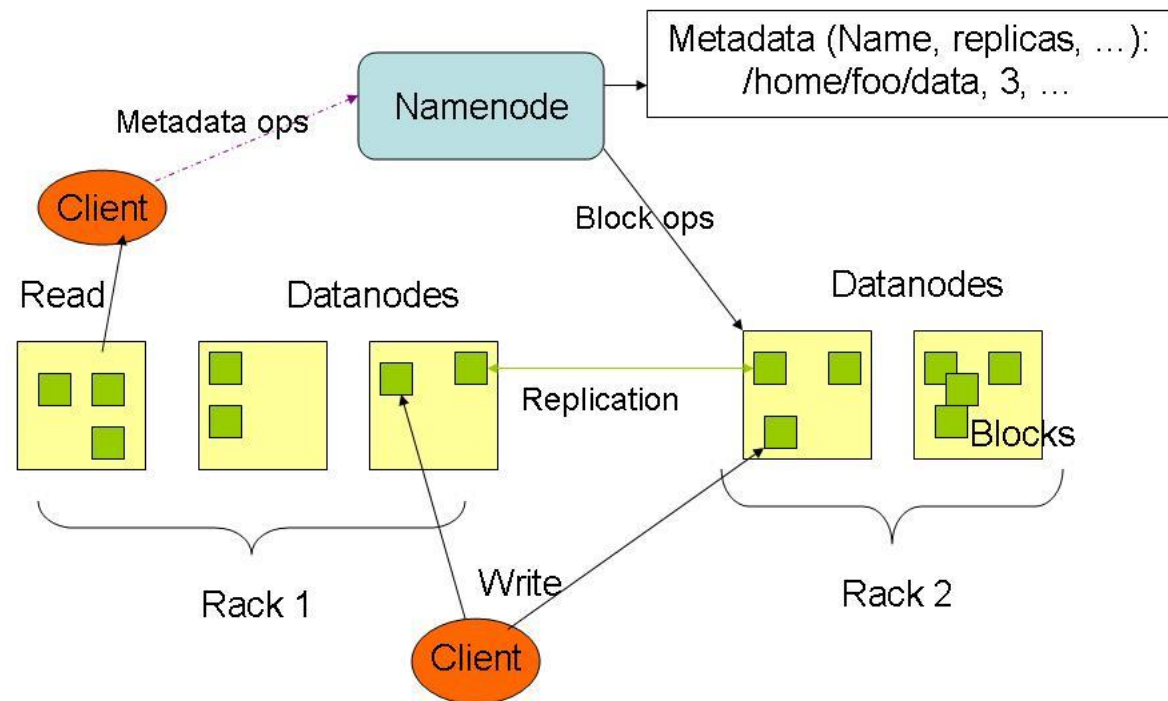
NameNode : Maps a file to a file-id and list of MapNodes

DataNode : Maps a block-id to a physical location on disk

SecondaryNameNode: Periodic merge of Transaction log

DataNodes

HDFS Architecture



FILE FORMAT

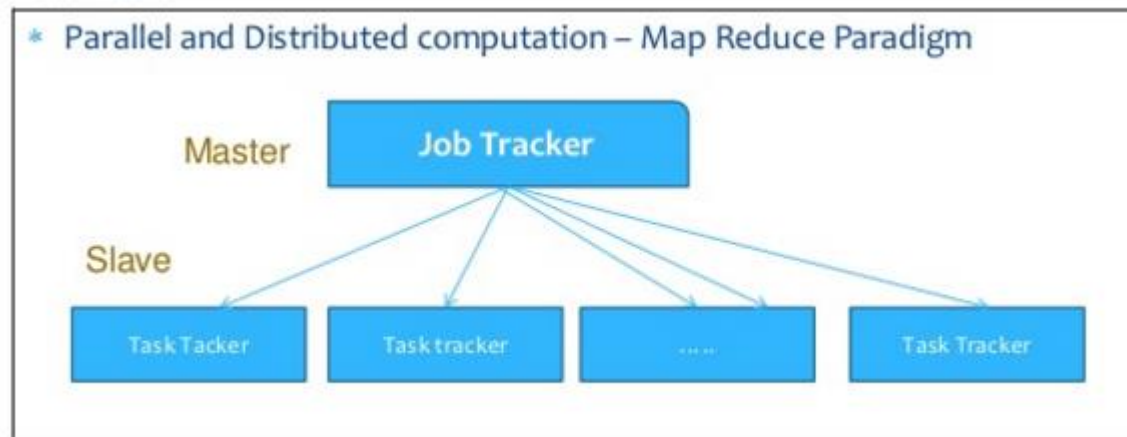
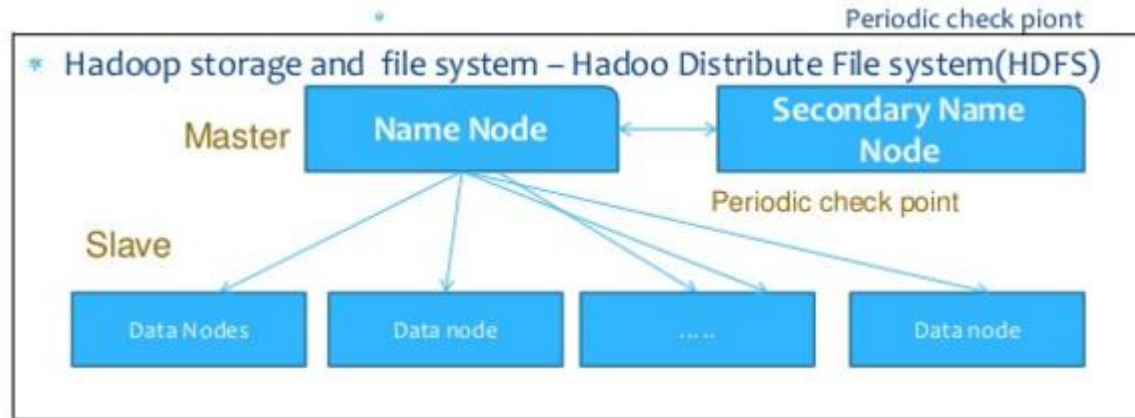
- Text / CSV
- JSON
- SequenceFile
- binary key/value pair format
- Avro
- Parquet
- ORC
- optimized row columnar format



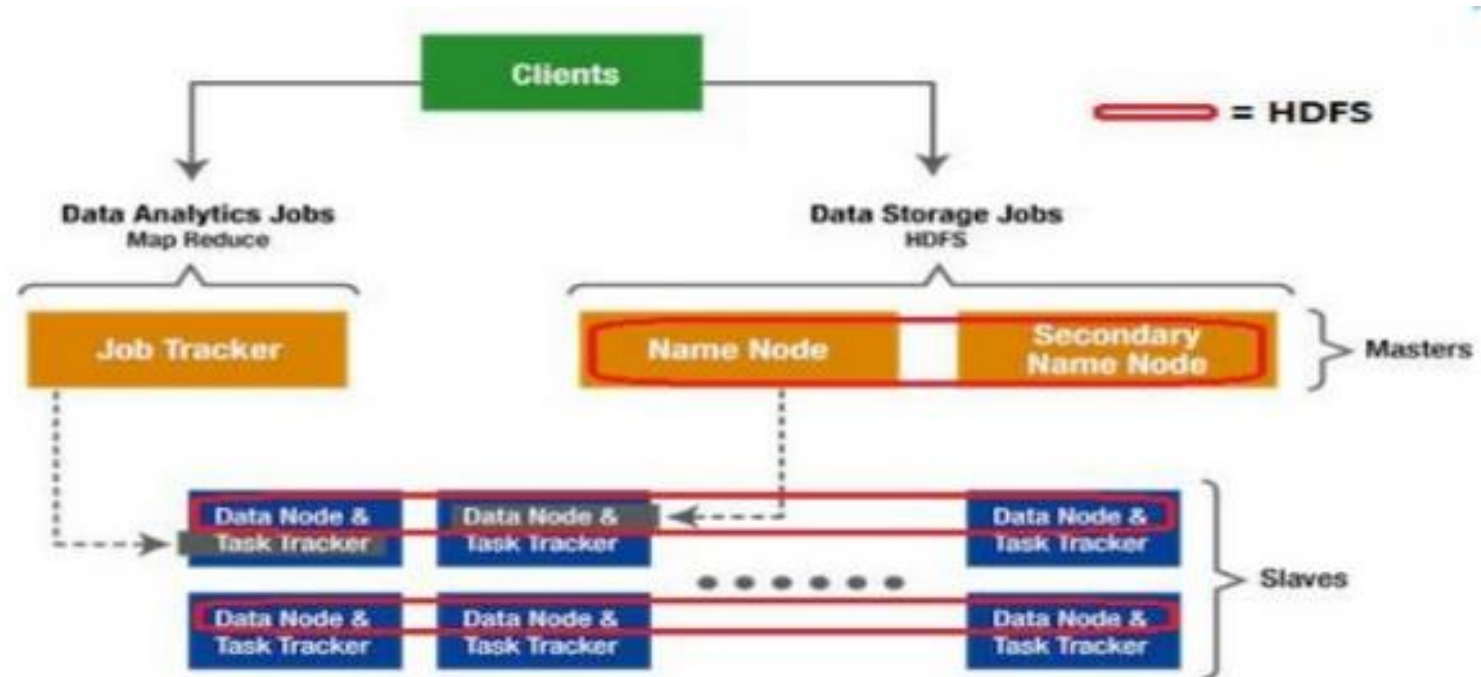
HADOOP



HADOOP



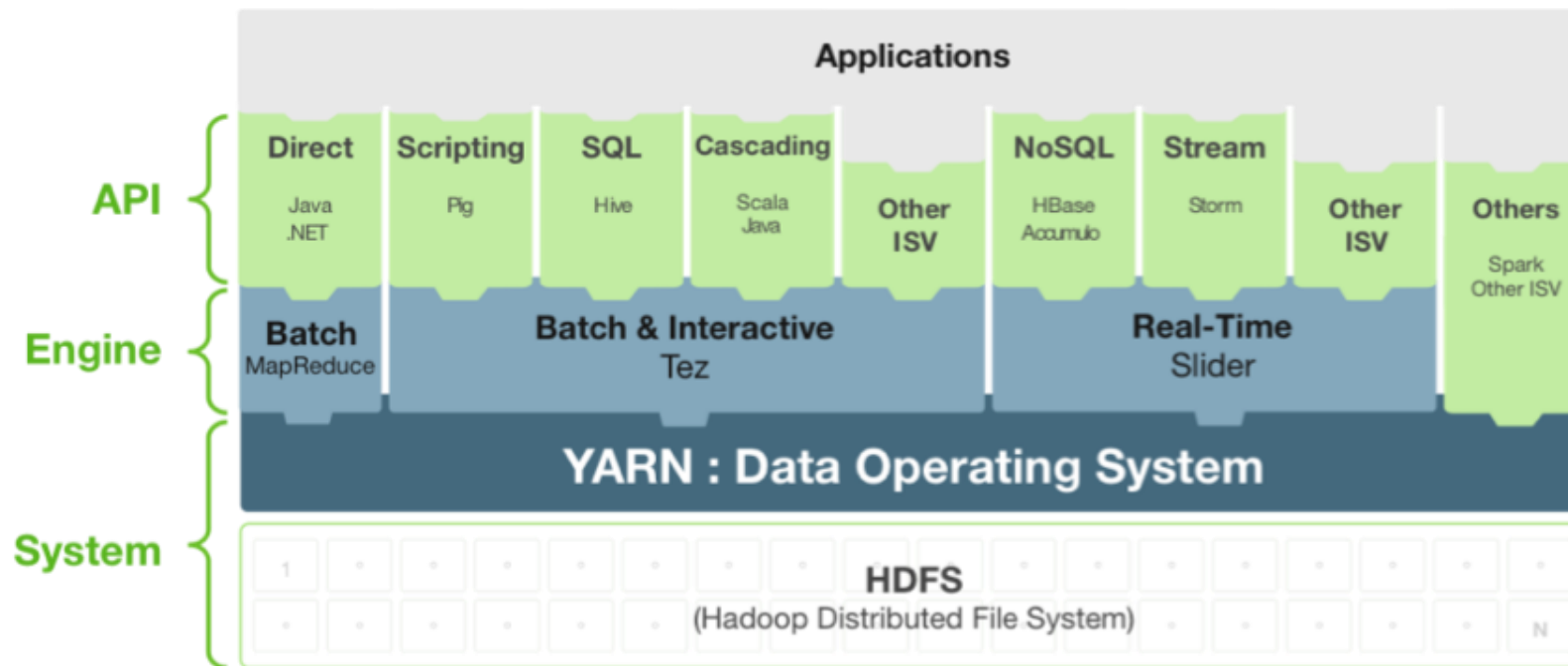
HADOOP ARCHITECTURE



HADOOP 2.0

Accumulo

- sorted, distributed key/value store is a robust, scalable, HP data storage and retrieval system.
- Hbase
 - distributed column-oriented database built on top of the Hadoop file system
- SPARK
 - in-memory data processing engine with development APIs in Scala, Java, and Python





MAP REDUCE



(PARALLEL) MAPREDUCE (I)

- MapReduce is a programming model for processing and generating large data sets with a parallel, distributed algorithm on a cluster
- Programmer defines two functions, map & reduce
 - $\text{Map}(k1, v1) \rightarrow \text{list}(k2, v2)$. Takes a series of key/value pairs, processes each, generates zero or more output key/value pairs
 - $\text{Reduce}(k2, \text{list}(v2)) \rightarrow \text{list}(v3)$. Executed once for each unique key $k2$ in the sorted order; iterate through the values associated with that key and produce zero or more outputs
- System “shuffles” data between map and reduce (so “reduce” function has set of data for its keys) automatically handles system failures, etc.

MAPREDUCE: WORD COUNT EXAMPLE (PSEUDOCODE)

```
map(String input_key, String input_value):
```

```
    // input_key: document name
```

```
    // input_value: document contents
```

```
    for each word w in input_value:
```

```
        EmitIntermediate(w, "1");
```

```
reduce(String output_key, Iterator intermediate_values):
```

```
    // output_key: a word
```

```
    // output_values: a list of counts
```

```
    int result = 0;
```

```
    for each v in intermediate_values:
```

```
        result += ParseInt(v);
```

```
    Emit(AsString(result));
```

Any 12
Ball 1
Computer 3
...

MAPREDUCE PROBLEMS & POTENTIAL SOLUTION

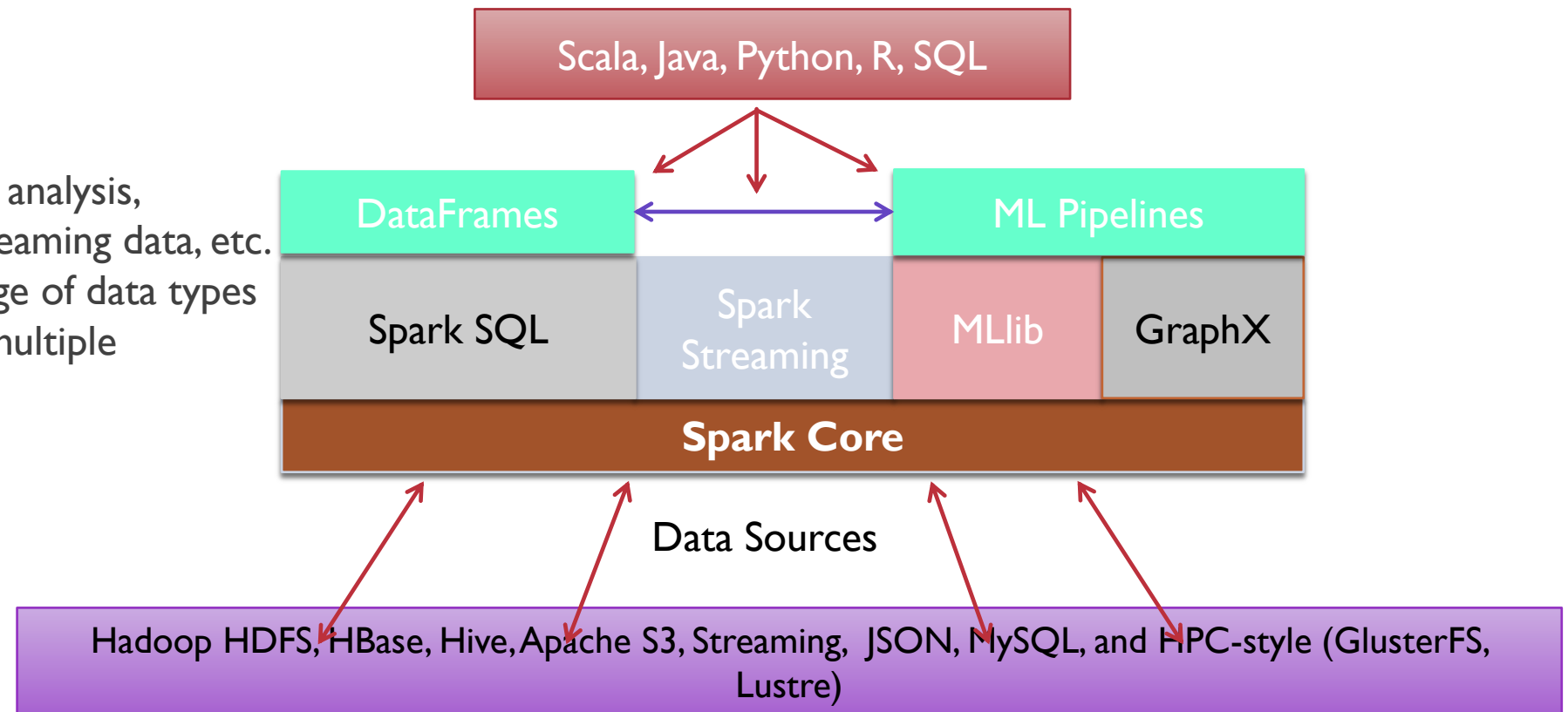
- MapReduce problems:
 - Many problems aren't easily described as map-reduce
 - Persistence to disk typically slower than in-memory work
- Alternative: Apache Spark
 - a general-purpose processing engine that can be used instead of MapReduce

APACHE SPARK

- Processing engine; instead of just “map” and “reduce”, defines a large set of *operations* (transformations & actions)
 - Operations can be arbitrarily combined in any order
- Open source software
- Supports Java, Scala and Python
- Key construct: Resilient Distributed Dataset (RDD)

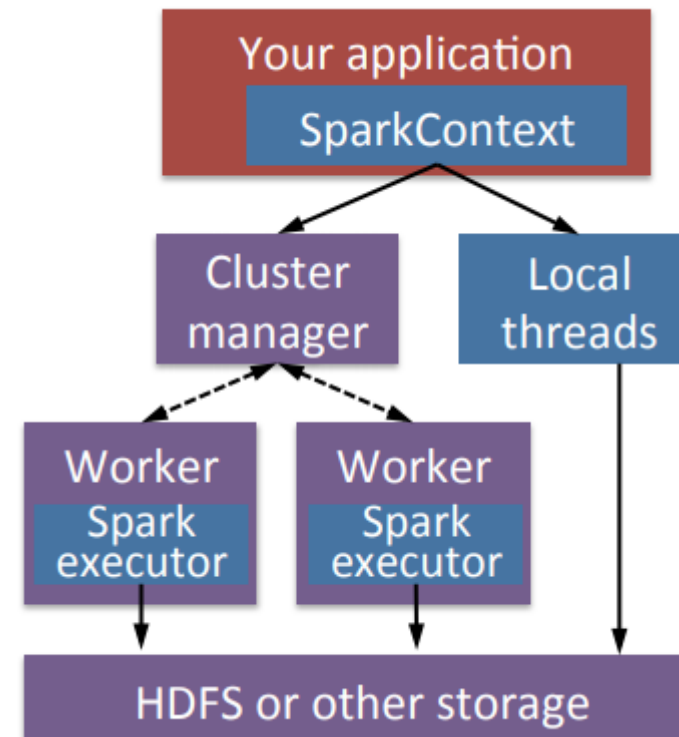
APACHE SPARK

Apache Spark supports data analysis, machine learning, graphs, streaming data, etc. It can read/write from a range of data types and allows development in multiple languages.



SOFTWARE COMPONENT

- Spark runs as a library in your program (1 instance per app)
- Runs tasks locally or on cluster
 - Mesos, YARN or standalone mode
- Accesses storage systems via Hadoop InputFormat API
 - Can use HBase, HDFS, S3, ...



LOCAL EXECUTION

- Just pass local or local[k] as master URL
- Debug using local debuggers
 - For Java / Scala, just run your program in a debugger
 - For Python, use an attachable debugger (e.g. PyDev)
- Great for development & unit tests

GRAY SORT COMPETITION: WINNER SPARK-BASED (PREVIOUSLY MR)

	Hadoop MR Record	Spark Record (2014)	Spark-based System 3x faster with 1/10 # of nodes
Data Size	102.5 TB	100 TB	
Elapsed Time	72 mins	23 mins	
# Nodes	2100	206	
# Cores	50400 physical	6592 virtualized	
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	
Sort rate	1.42 TB/min	4.27 TB/min	
Sort rate/node	0.67 GB/min	20.7 GB/min	

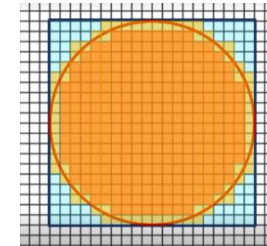
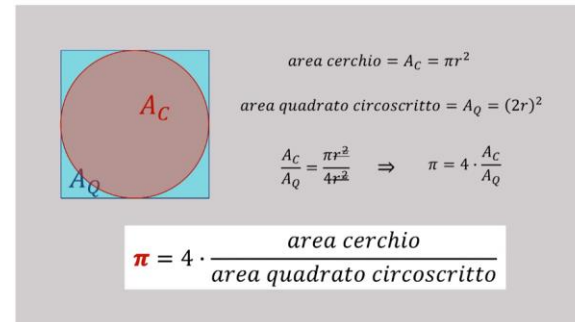
Sort benchmark, Daytona Gray: sort of 100 TB of data (1 trillion records)

<http://databricks.com/blog/2014/11/05/spark-officially-sets-a-new-record-in-large-scale-sorting.html>

SPARK EXAMPLE #2 (PYTHON)

```
# Estimate  $\pi$  (compute-intensive task).  
# Pick random points in the unit square  $((0, 0)$  to  $(1, 1))$ ,  
# See how many fall in the unit circle. The fraction should be  $\pi / 4$   
# Note that “parallelize” method creates an RDD  
def sample(p):  
    x, y = random(), random()  
    return 1 if x*x + y*y < 1 else 0
```

```
count = spark.parallelize(xrange(0, NUM_SAMPLES)).map(sample) \  
    .reduce(lambda a, b: a + b)  
print "Pi is roughly %f" % (4.0 * count / NUM_SAMPLES)
```



SPARK EXAMPLE #3 (PYTHON)

```
# Logistic Regression - iterative machine learning algorithm
# Find best hyperplane that separates two sets of points in a
# multi-dimensional feature space. Applies MapReduce operation
# repeatedly to the same dataset, so it benefits greatly
# from caching the input in RAM
```

```
points = spark.textFile(...).map(parsePoint).cache()
w = numpy.random.rand(size = D) # current separating plane
for i in range(ITERATIONS):
    gradient = points.map(
        lambda p: (1 / (1 + exp(-p.y*(w.dot(p.x)))) - 1) * p.y * p.x
    ).reduce(lambda a, b: a + b)
    w -= gradient
print "Final separating plane: %s" % w
```

SPARK EXAMPLE #3 (JAVA)

```
// Same thing in Java
class ComputeGradient extends Function<DataPoint, Vector> {
    private Vector w;
    ComputeGradient(Vector w) { this.w = w; }
    public Vector call(DataPoint p) {
        return p.x.times(p.y * (1 / (1 + Math.exp(w.dot(p.x))) - 1));
    }
}

JavaRDD<DataPoint> points = spark.textFile(...).map(new ParsePoint()).cache();
Vector w = Vector.random(D); // current separating plane
for (int i = 0; i < ITERATIONS; i++) {
    Vector gradient = points.map(new ComputeGradient(w)).reduce(new AddVectors());
    w = w.subtract(gradient);
}
System.out.println("Final separating plane: " + w);
```

APACHE SPARK: LIBRARIES “ON TOP” OF CORE THAT COME WITH IT

- Spark SQL
- Spark Streaming – stream processing of live datastreams
- MLlib - machine learning
- GraphX – graph manipulation
 - extends Spark RDD with Graph abstraction: a directed multigraph with properties attached to each vertex and edge.

DATAFRAMES & SPARKSQL

- DataFrames (DFs) is one of the other distributed datasets organized in named columns
- Similar to a relational database, Python Pandas Dataframe or R's DataTables
 - Immutable once constructed
 - Track lineage
 - Enable distributed computations
- How to construct Dataframes
 - Read from file(s)
 - Transforming an existing DFs(Spark or Pandas)
 - Parallelizing a python collection list
 - Apply transformations and actions

DATAFRAME EXAMPLE

- `// Create a new DataFrame that contains “students”`
- `students = users.filter(users.age < 21)`

- `//Alternatively, using Pandas-like syntax`
- `students = users[users.age < 21]`

- `//Count the number of students users by gender`
- `students.groupBy("gender").count()`

- `// Join young students with another DataFrame called logs`
- `students.join(logs, logs.userId == users.userId,`
- `“left_outer”)`



DATA LAKE ARCHITECTURE

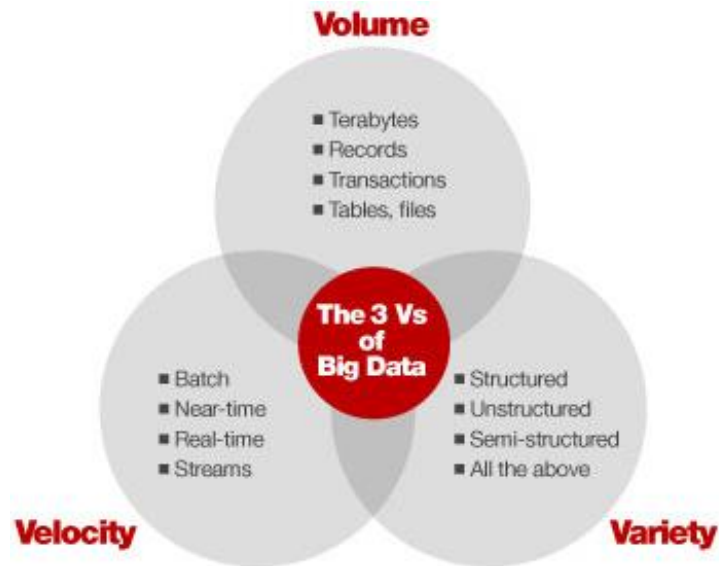


OUTLINE

- Definition
- Architecture for data lake
- Software component

HISTORY

- The term “data lake” was coined 2010 by James Dixon
- He distinguished between the approach to manage data on
 - Hadoop and data marts /warehouses:
- data mart is a store of bottled water
 - cleansed and packaged and structured for easy consumption
- The data lake is a large body of water in a more natural state.
 - The contents of the data lake stream in from a source to fill the lake,
 - users of the lake can come to examine, dive in, or take samples.



BIG DATA

- Nuove architetture per la gestione delle informazioni
- Dal magazzino di dati al lago di dati



- Radicale cambiamento della modalità di analisi e interrogazione delle informazioni
- Da analisi basate sui dati disponibili nel warehouse (data-driven)
- Analisi basate sulle richieste dell'utente finale (demand-driven)
- Cambiamento nell'interazione fra utente finale e dati

NEEDS

- Enterprise Data Warehouse (EDW) are rigid, very well structured data models
- New sources
 - Volume
 - Variety
 - Velocity (less relevant, but not always)
- Licence costs
- Change of paradigm

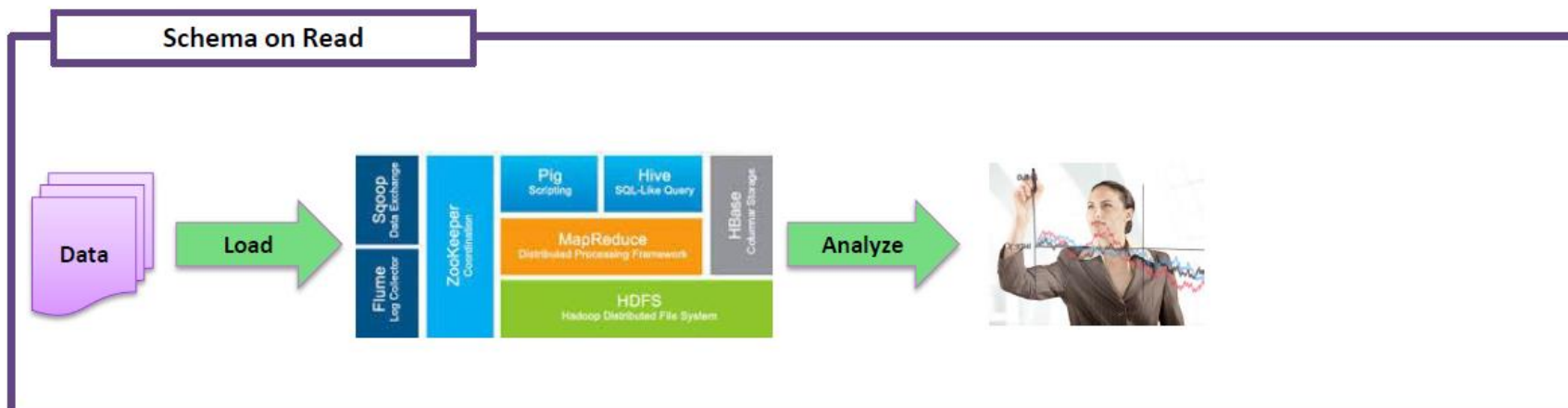
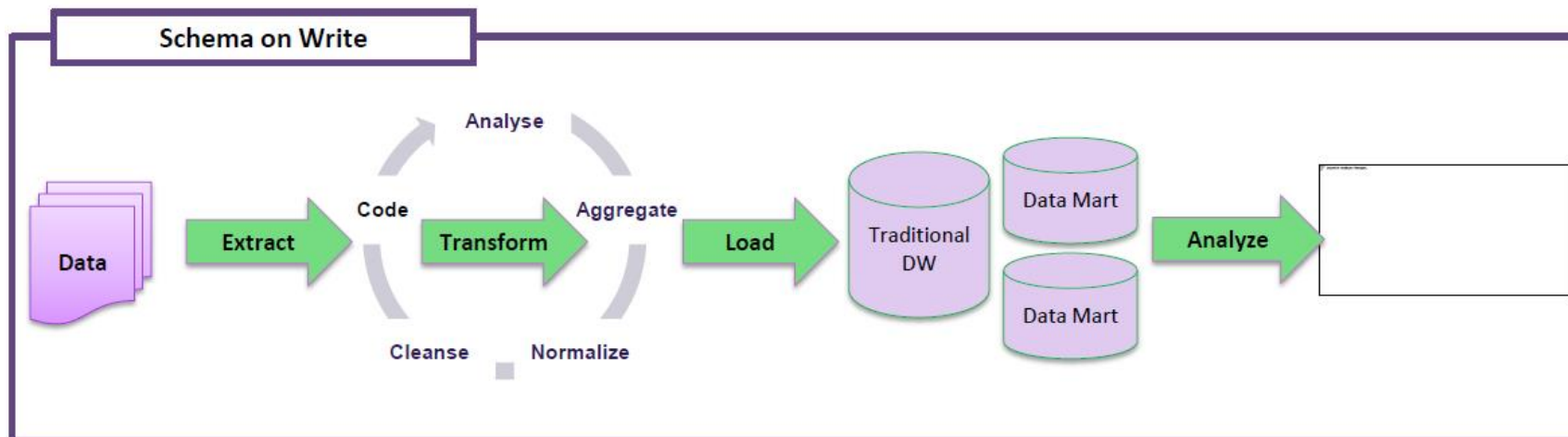
Table 1-1. Differences between EDWs and data lakes

Attribute	EDW	Data lake
Schema	Schema-on-write	Schema-on-read
Scale	Scales to large volumes at moderate cost	Scales to huge volumes at low cost
Access methods	Accessed through standardized SQL and BI tools	Accessed through SQL-like systems, programs created by developers, and other methods
Workload	Supports batch processing, as well as thousands of concurrent users performing interactive analytics	Supports batch processing, plus an improved capability over EDWs to support interactive queries from users
Data	Cleansed	Raw
Complexity	Complex integrations	Complex processing
Cost/efficiency	Efficiently uses CPU/IO	Efficiently uses storage and processing capabilities at very low cost
Benefits	• Transform once, use many • Clean, safe, secure data • Provides a single enterprise-wide view of data from multiple sources • Easy to consume data • High concurrency • Consistent performance • Fast response times	• Transforms the economics of storing large amounts of data • Supports Pig and HiveQL and other high-level programming frameworks • Scales to execute on tens of thousands of servers • Allows use of any tool • Enables analysis to begin as soon as the data arrives • Allows usage of structured and unstructured content from a single store • Supports agile modeling by allowing users to change models, applications, and queries

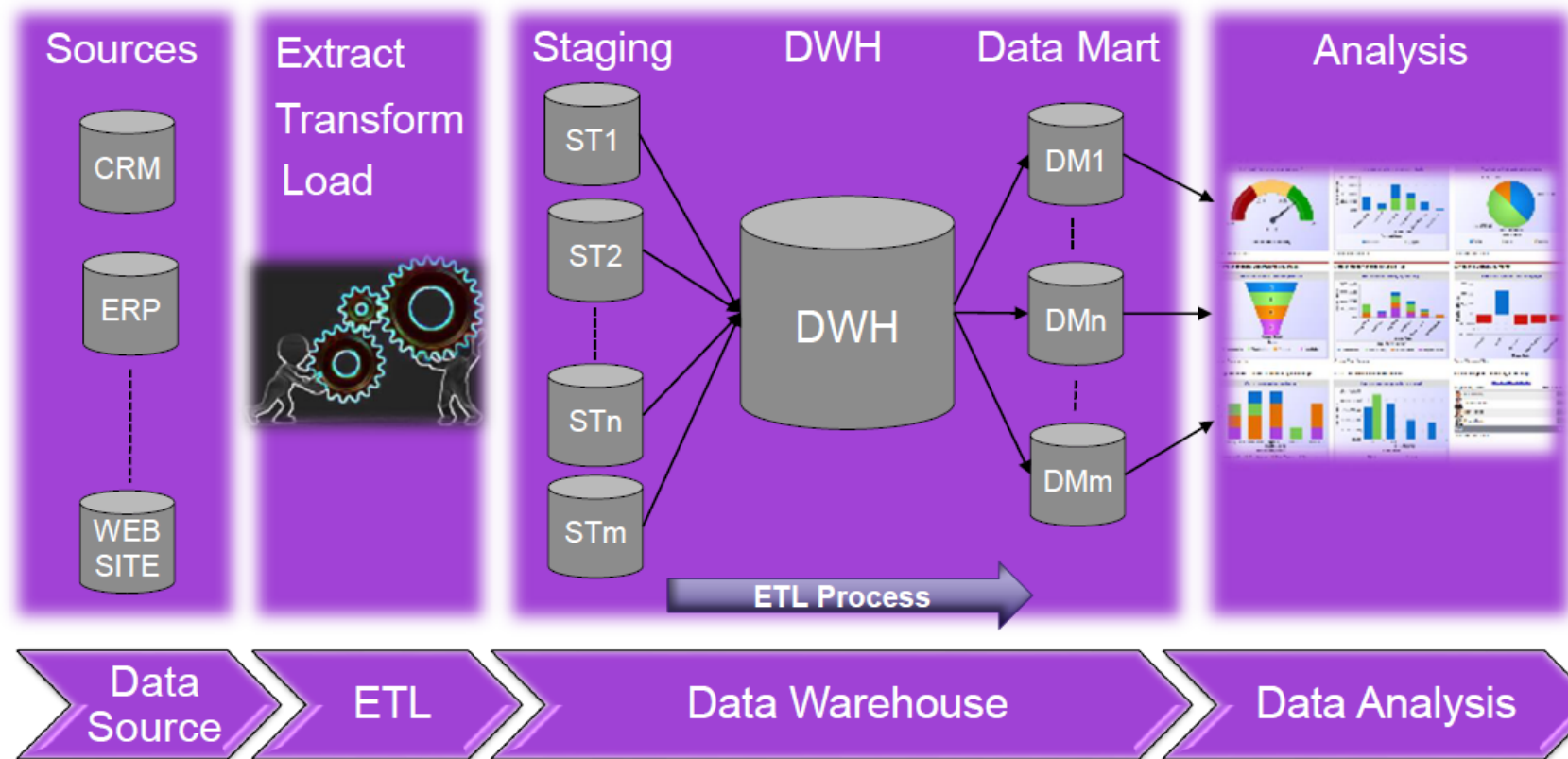
DATA LAKE

- To realize a bottom up approach there is no need to define the schema of data before to load data
- Data must be modelled according to the task
- In EDW the task is strongly related to how the data are modelled

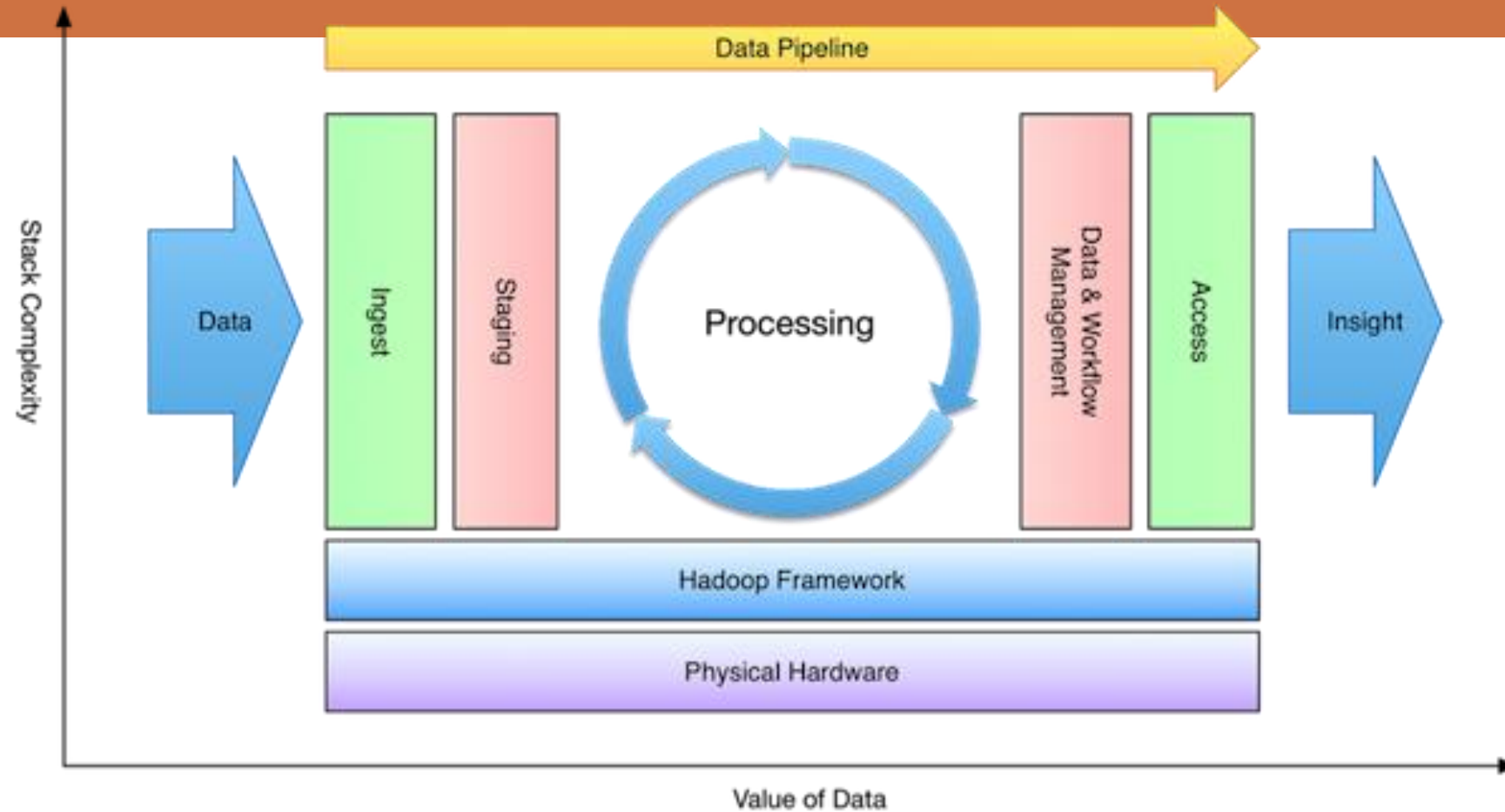
WHEN WRITE?



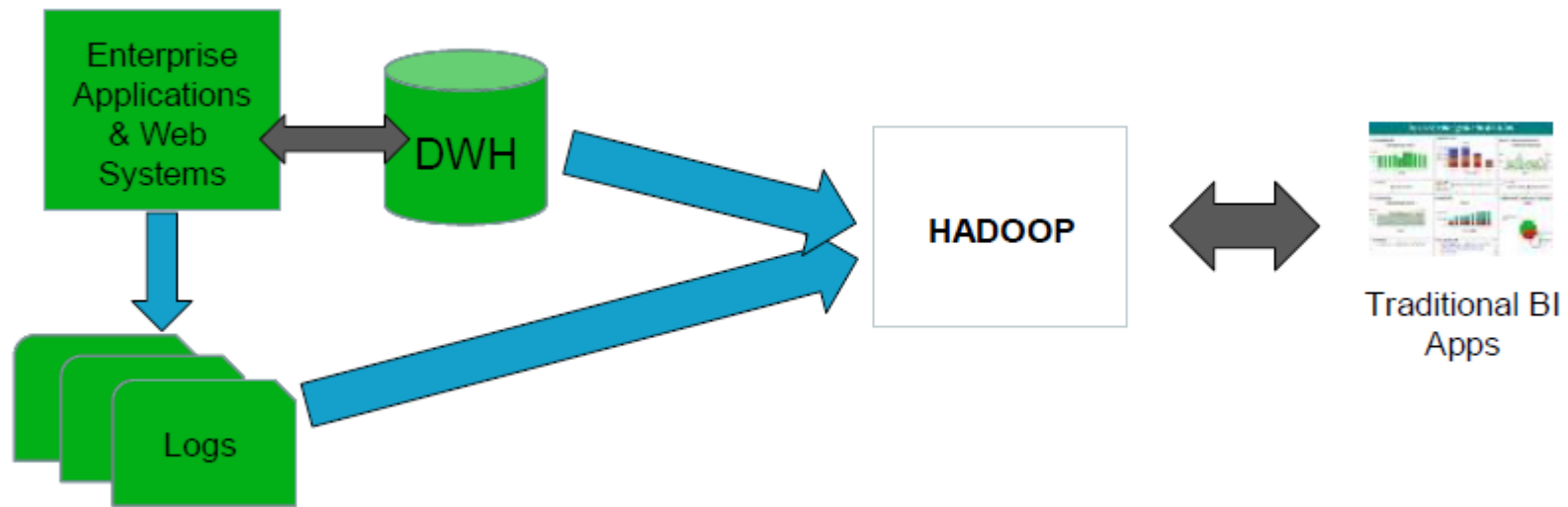
DW ARCHITECTURE



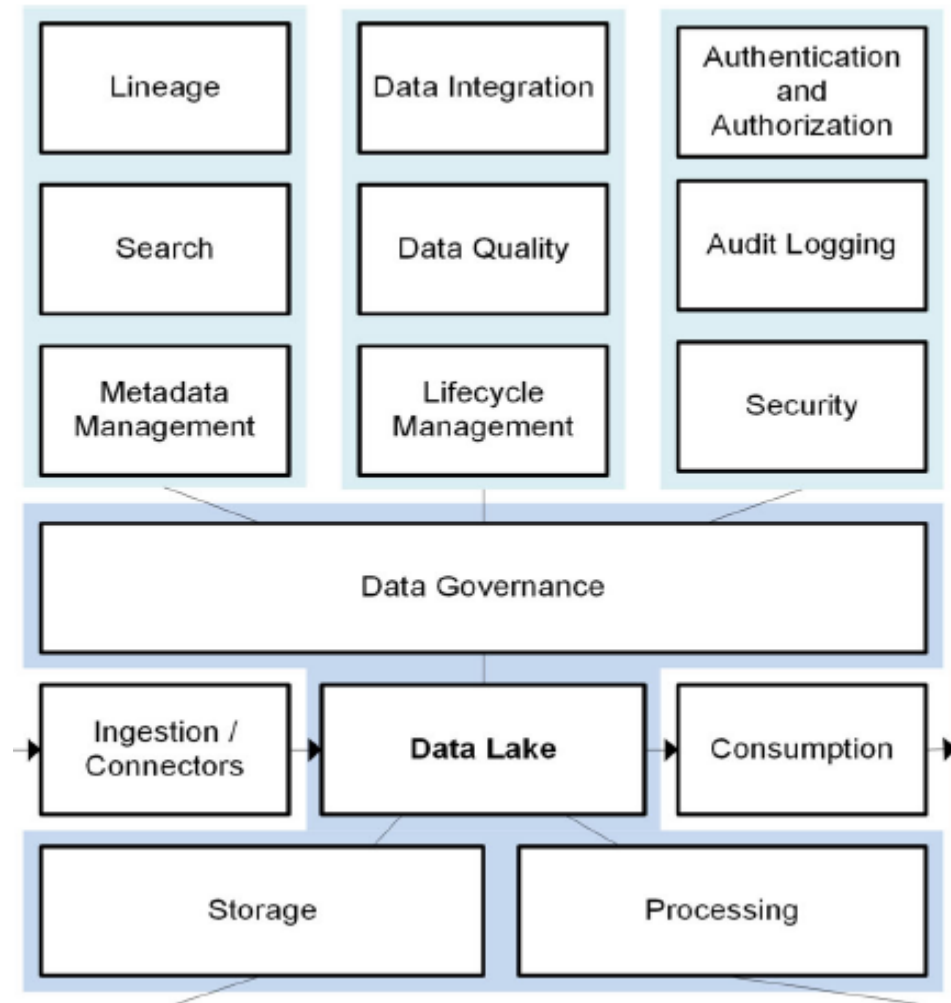
HADHOOP BASED ARCHITECTURE



HOW TO INTEGRATE EDW AND DATALAKE?



DATA LAKE COMPONENT ARCHITECTURE





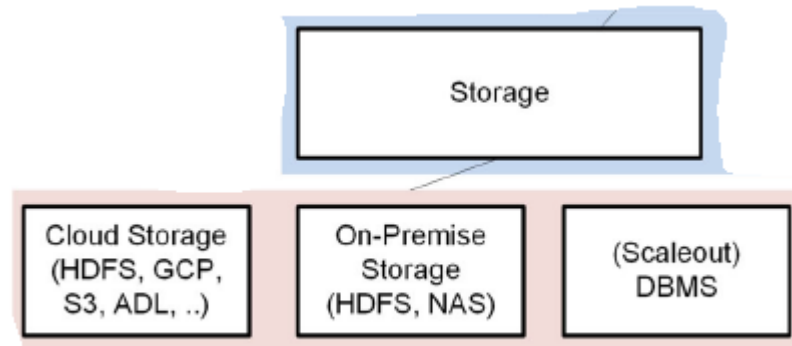
STORAGE



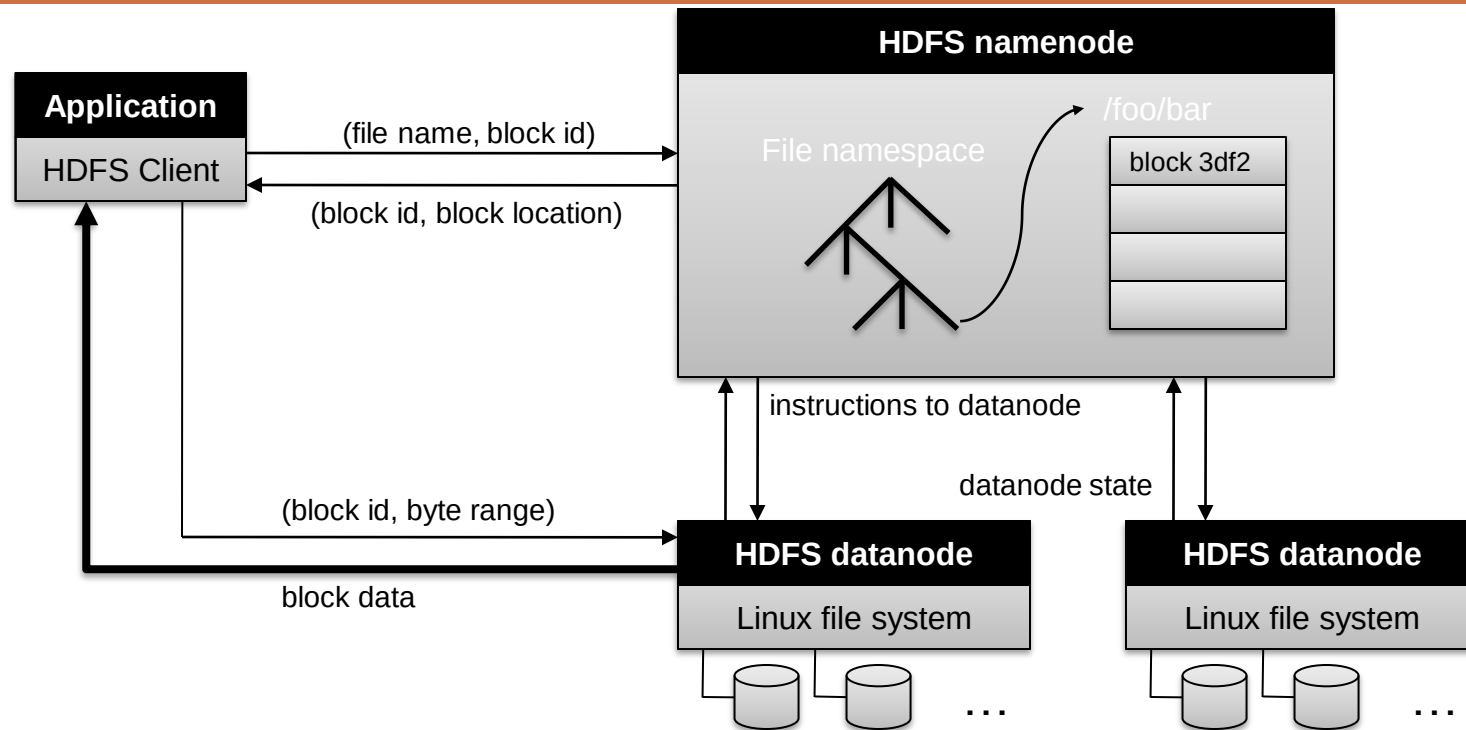
STORAGE

- What is the best solution for
 - Any type, any size, any rate?
- File system!
 - A distributed file system → Hadoop Distributed File System (HDFS)
- And NoSQL engine?
 - Any type (no)?
 - Any size (yes)
 - Any rate (maybe)

DATA STORAGE



HDFS

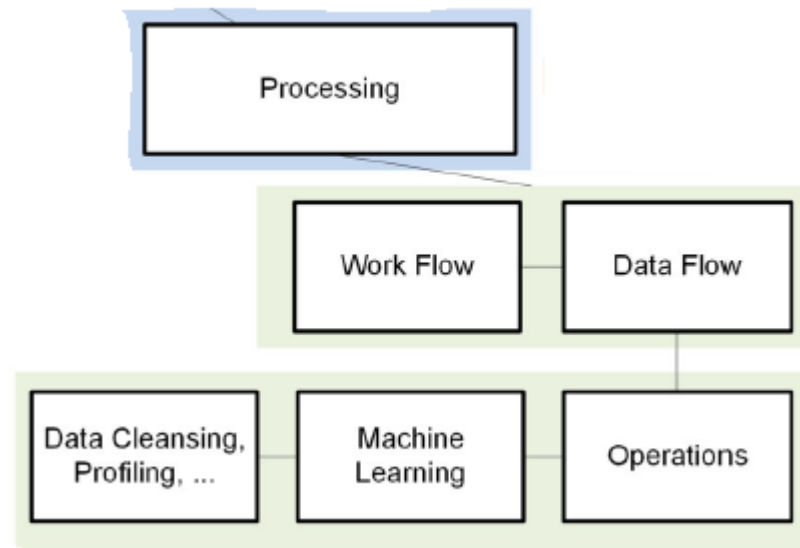




DATA PROCESSING



PROCESSING

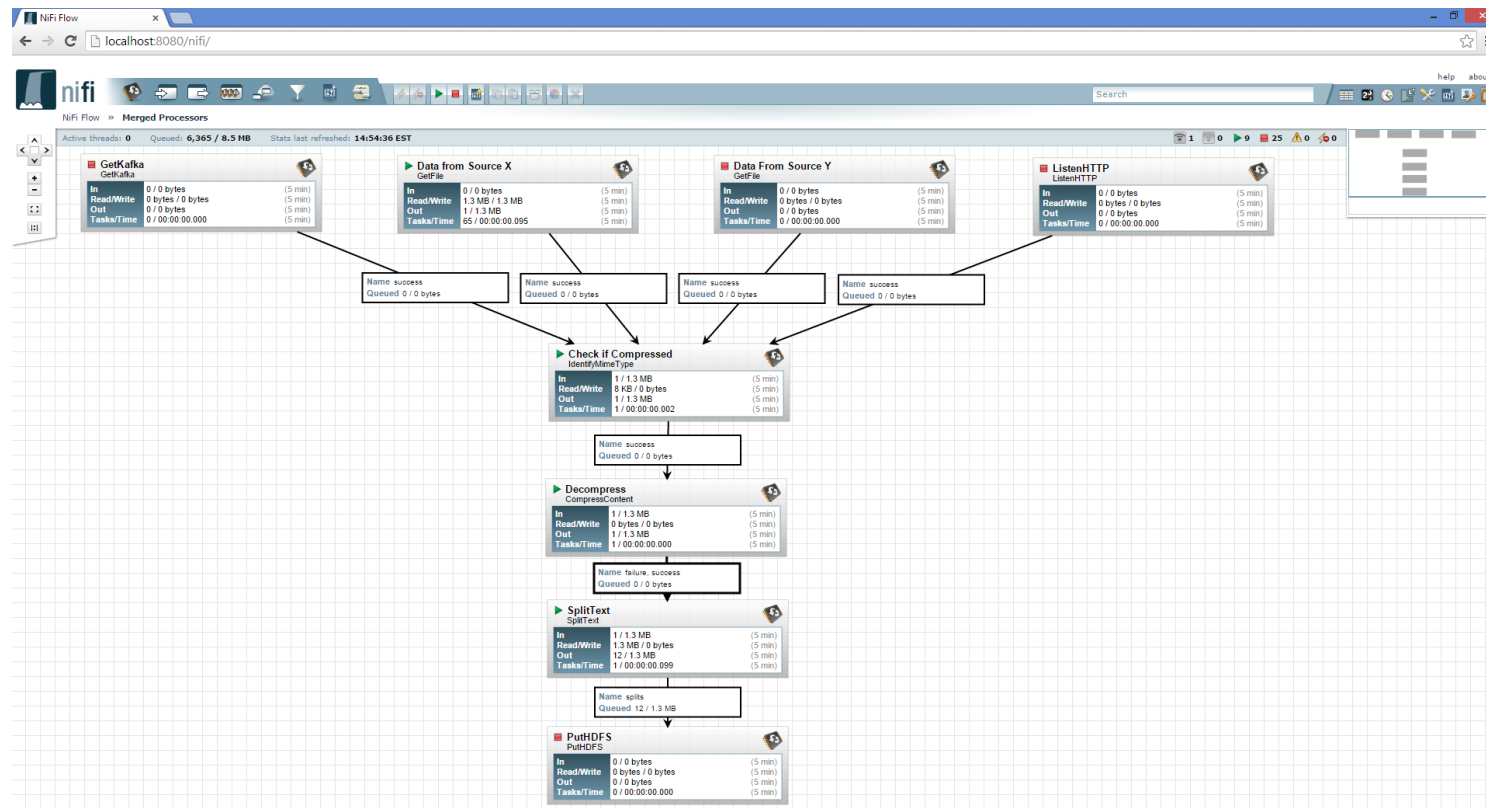


PROCESSING

- Three stages to extract information from the data stored on the lake
 - data preparation (see data integration),
 - data analytics (or machine learning),
 - Result provisioning for consumption.
- How do you perform them?
- General purpose processing frameworks
 - Hadoop MapReduce, Apache Spark, Flink or Storm
 - (micro-)batch (MapReduce, Spark) or stream-based (Flink, Storm)

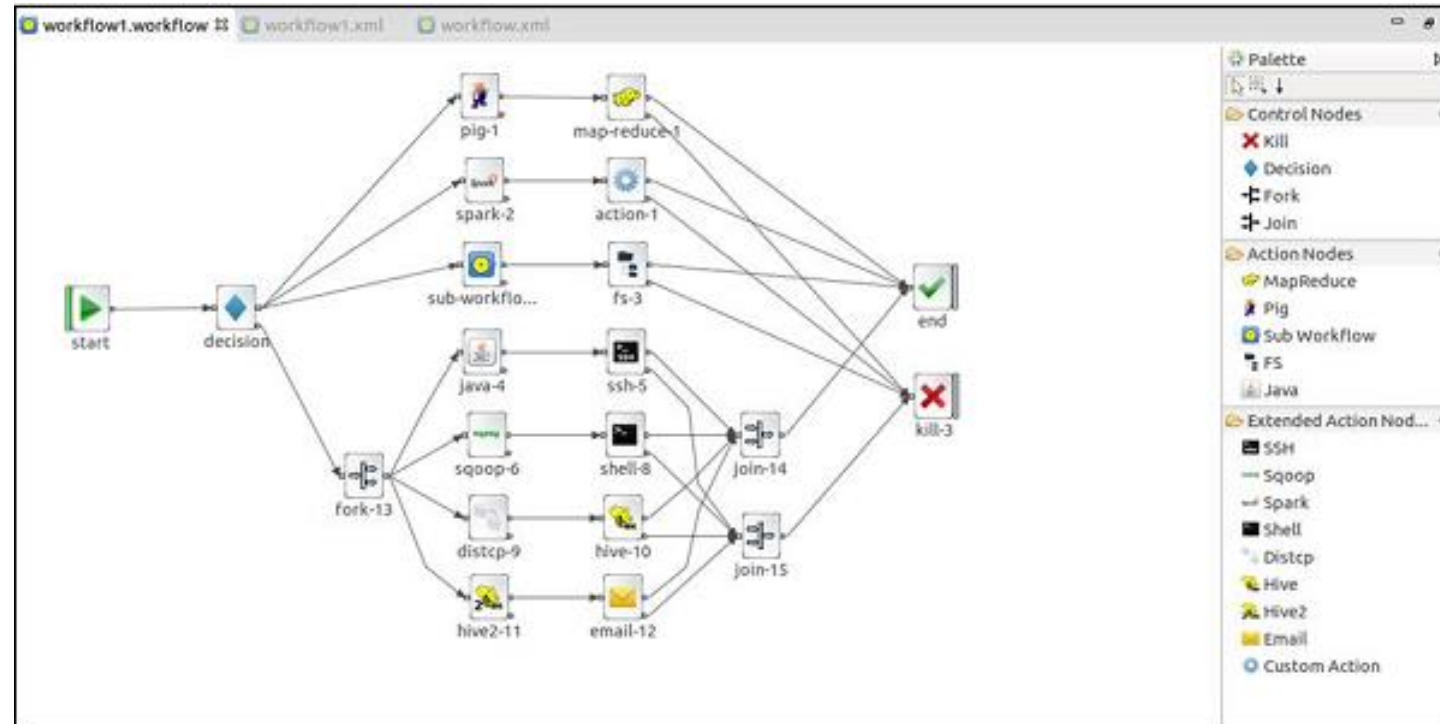
DATA FLOW

- To execute a set of data manipulation activities
- Apache NiFi



WORK FLOW

- To orchestrate several unit of works
- Apache Oozie, Falcon



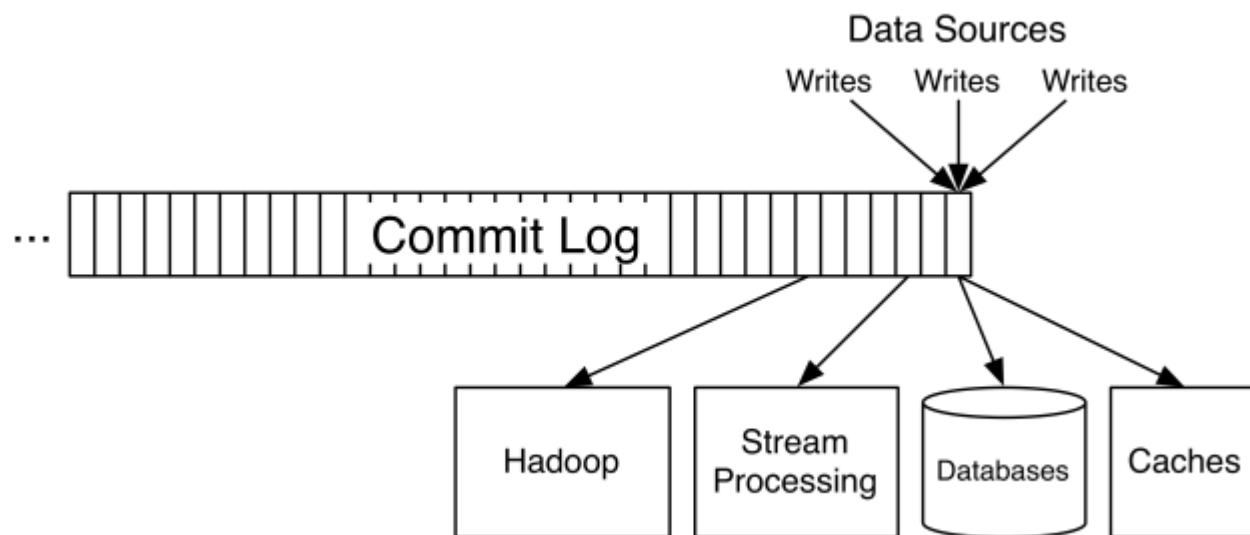
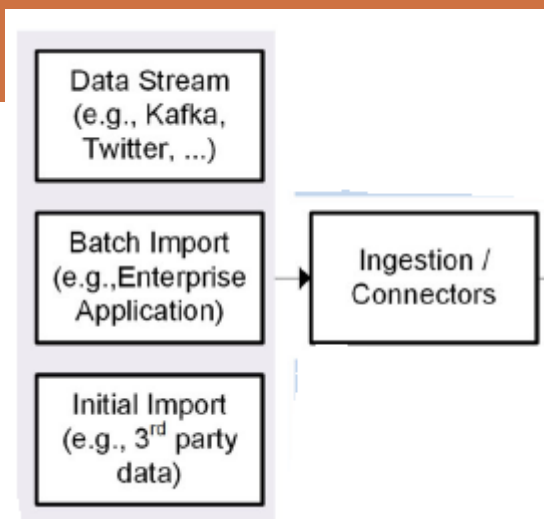


DATA INGESTION

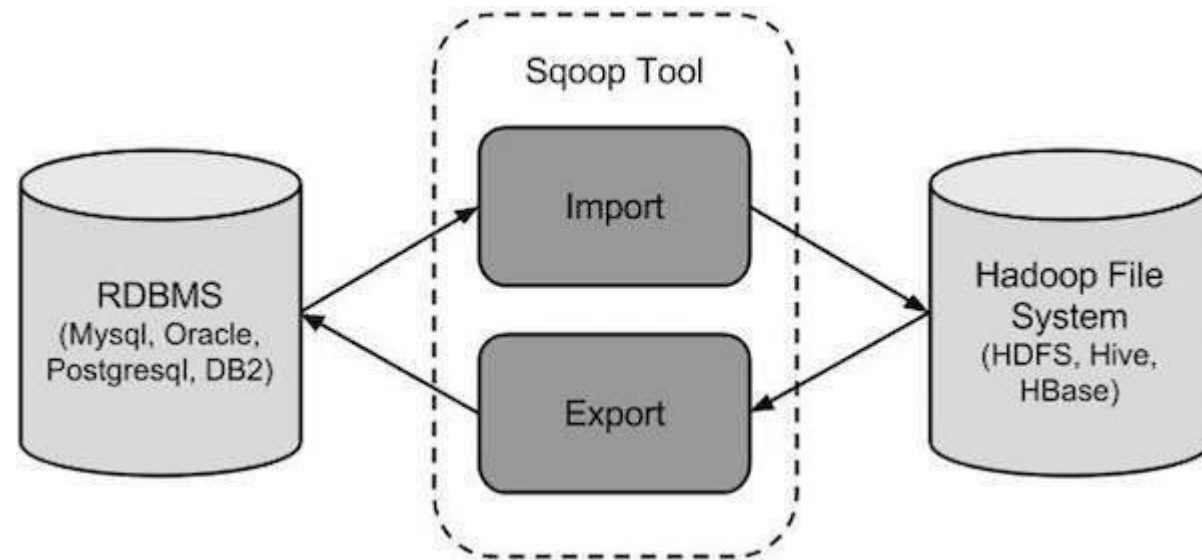
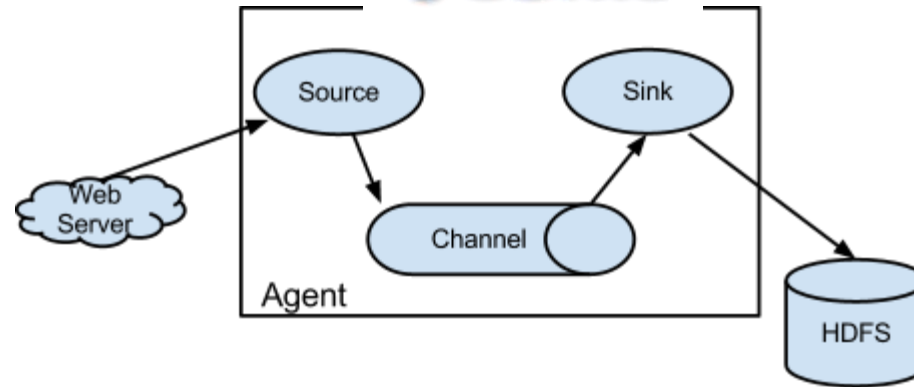


Apache Kafka

A high-throughput distributed messaging system.



CONNECTORS

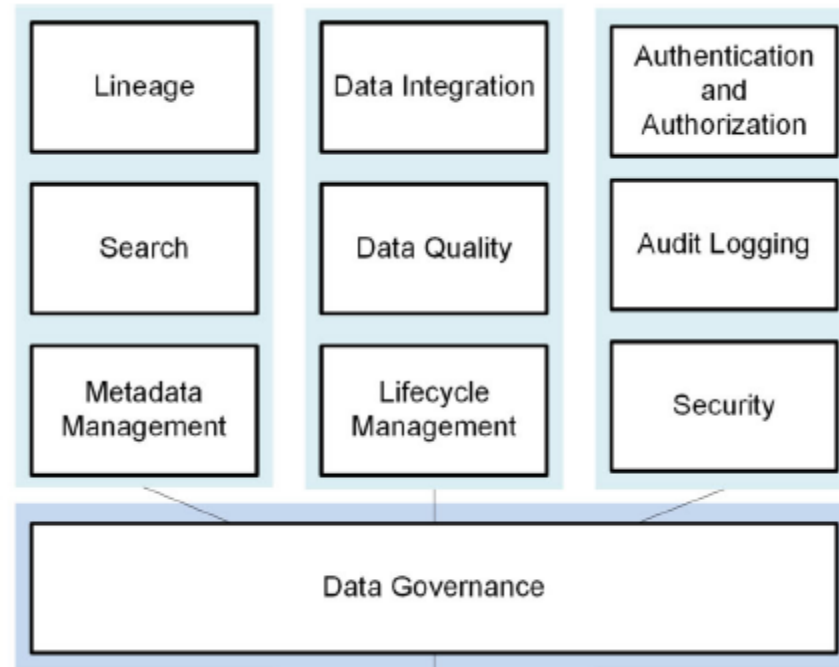




DATA GOVERNANCE



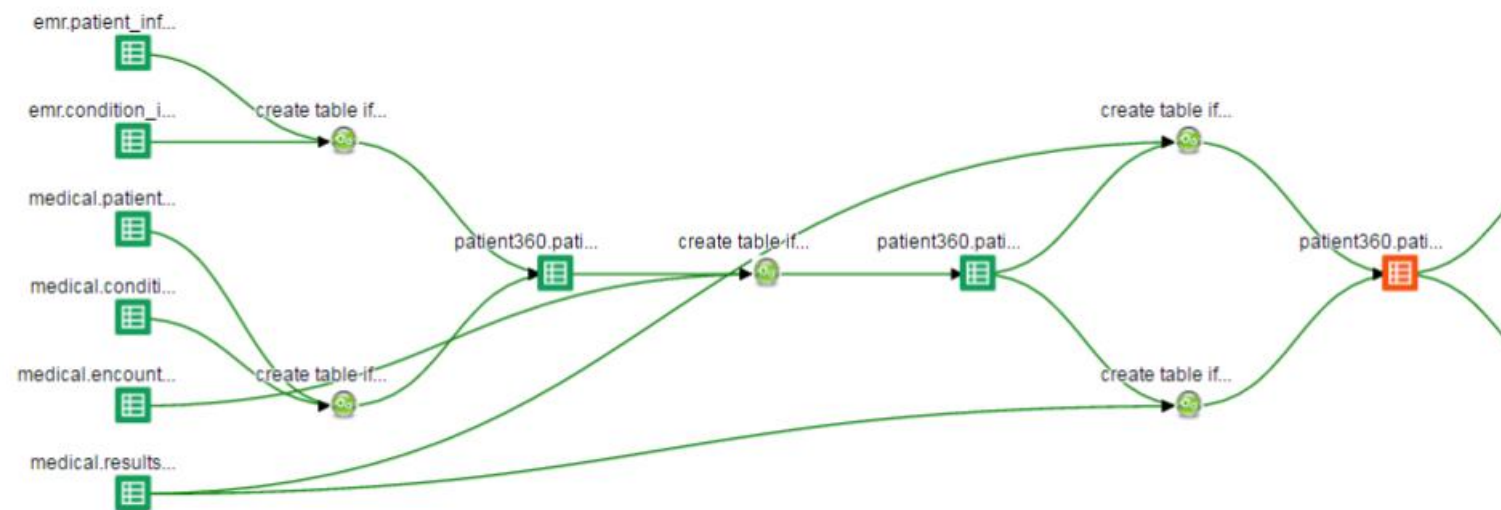
DATA GOVERNANCE



DATA LINEAGE

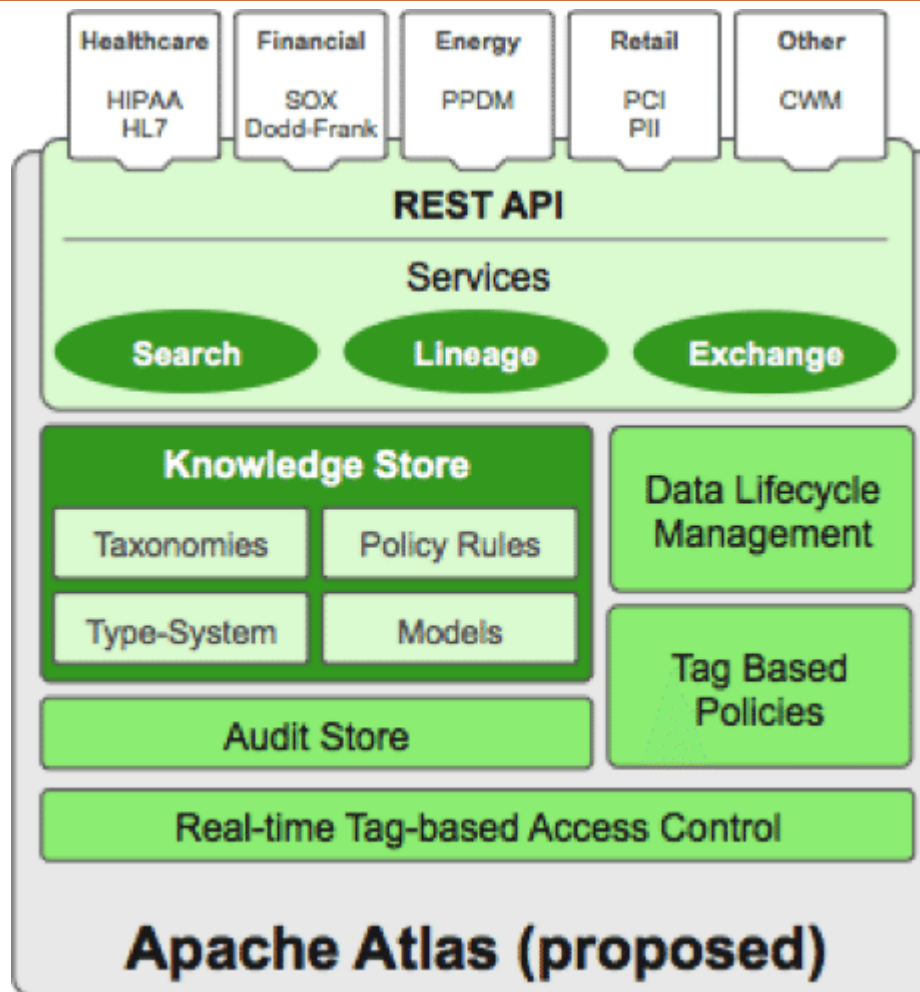
Lineage

Reset

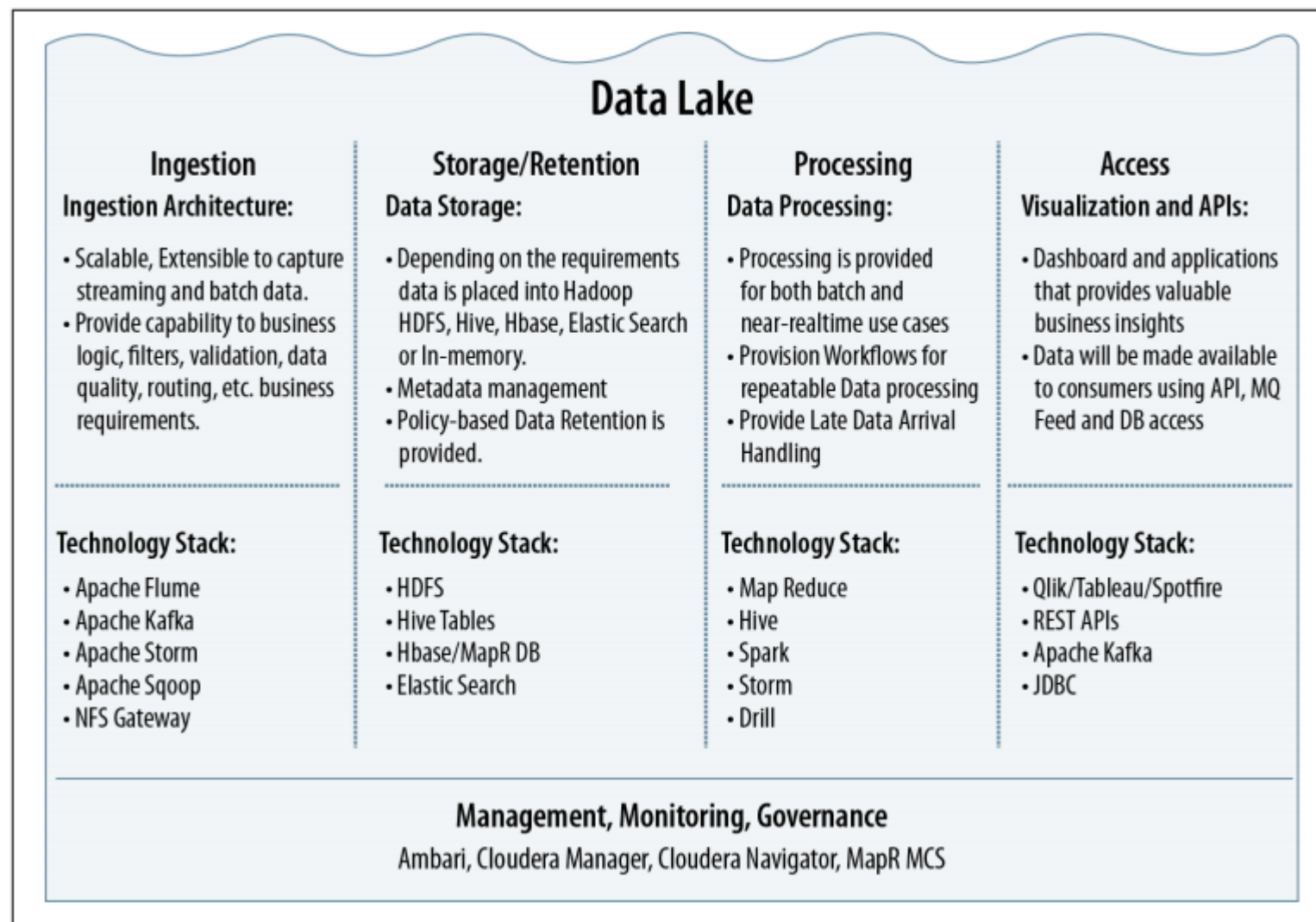


METADATA MANAGMENT

- Apache atlas
- Knowledge Graph

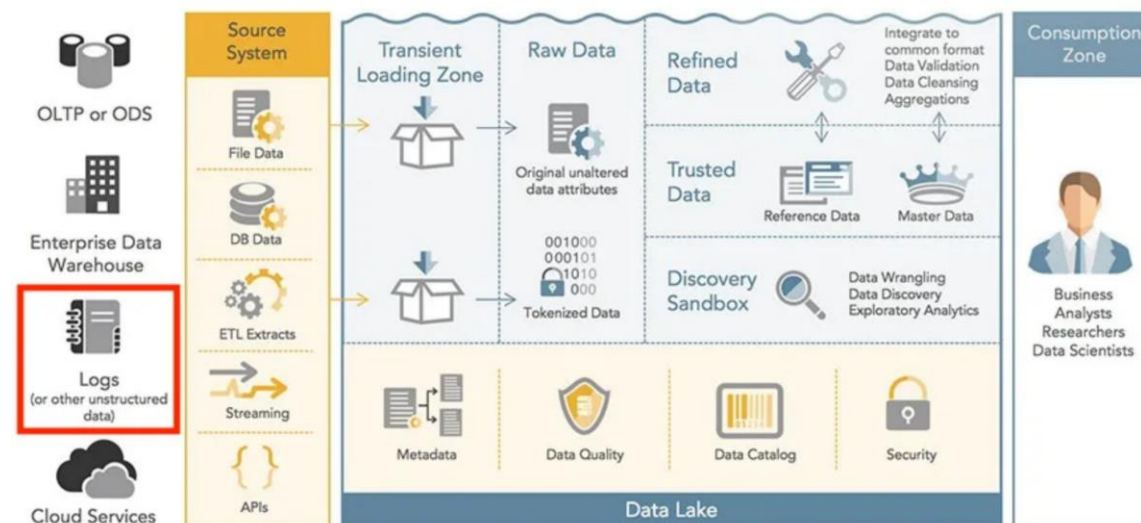


IN SUMMARY

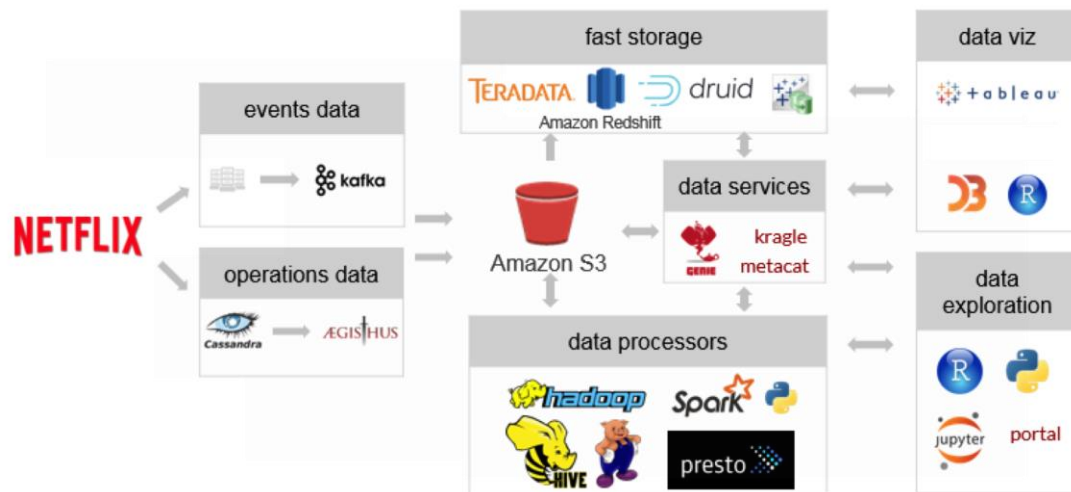


ANOTHER DATA LAKE ARCHITECTURE

- Different conceptual zone
- Raw zone (ODS)
- Refined zone (the kitchen)
- Truster data
- Discovery sandbox



NETFLIX



- Both "static" and dynamic data collection platform
- Different data storage systems for different information flows
- Different data analysis systems for different requests
- Analysis, visualization and exploration modes that depend on user requests

LIMITS

- Bigger is better
- Approach to data analysis
- Lack of systems for managing metadata
- Objective: smart data
 - Information suited to requests
 - Quality
 - Time
 - Level of depth and understanding



IS BIGGER BETTER?



UNIVERSITY OF MINNESOTA

EXPERTS @ MINNESOTA

EXPERIMENTAL EVALUATION OF INFORMATION OVERLOAD IN A PRODUCTION ENVIRONMENT.

Norman L. Chervany; Gary W. Dickson
(Profiled Author: Norman L. Chervany)
Management Science. 1974;20(10):1335-1344.

1974 experiment on a class of MBA students divided into two groups

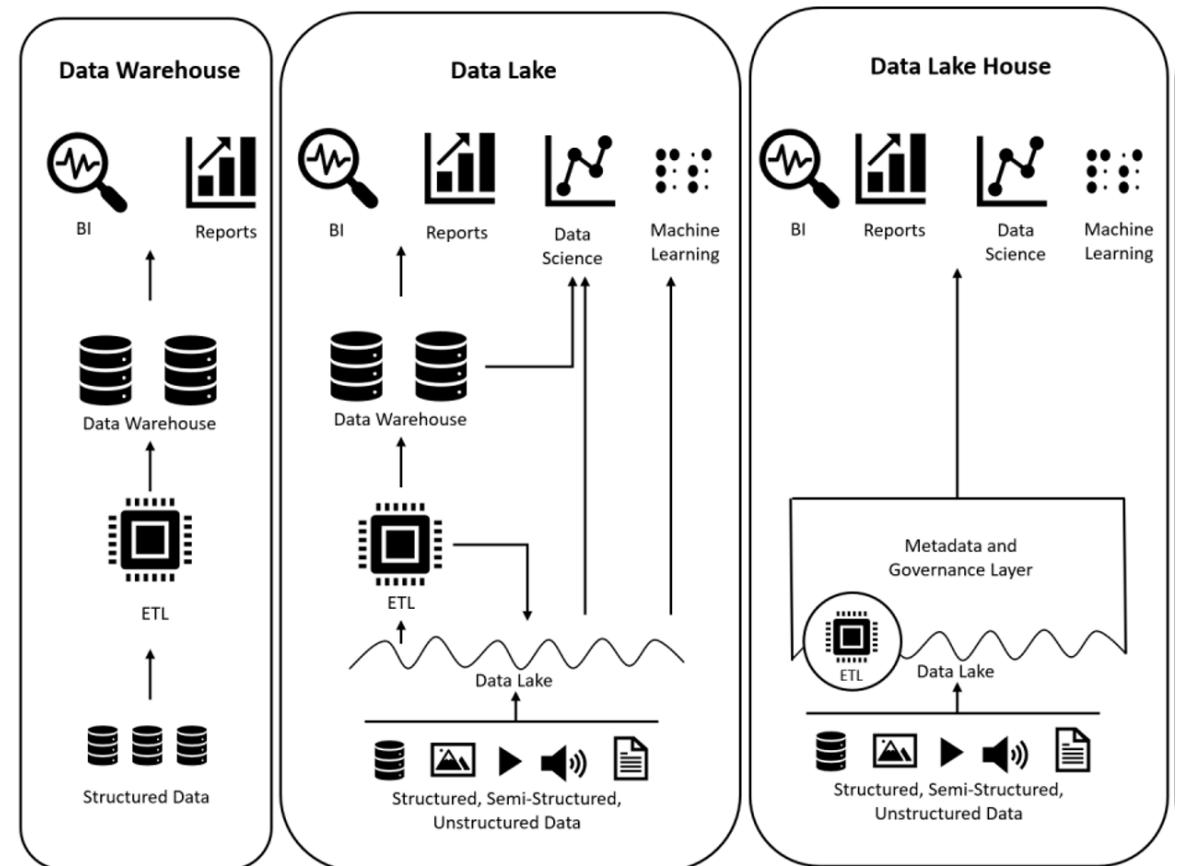
- The first group receives all information at the highest level of detail
 - Hypothesize a solution with a high level of confidence
 - The second group receives the same information, but aggregated
 - Provides a solution with a lower confidence level
-
- The second group provides the right solution
 - Having more data does not mean increasing the ability to make better decisions
 - Information overloading

APPROACH TO DATA ANALYSIS

- Study of the DNA of populations affected by hereditary diseases to identify the gene that causes the disease or particular human characteristics
- Comparison between DNA from different populations of individuals
- Genome-wide association studies (GWAS)
- Standard genetic methods have long estimated characteristics such as height to be 80% hereditary
- None of the genes were found
- If genes have such strong effects, where are they?
- The causes are more complex than a single change in a gene
- More complex interpretative models

FROM DATA WAREHOUSE TO DATA LAKEHOUSE

- From rigid architecture
- To more flexible approach
- Mix the two previous approaches

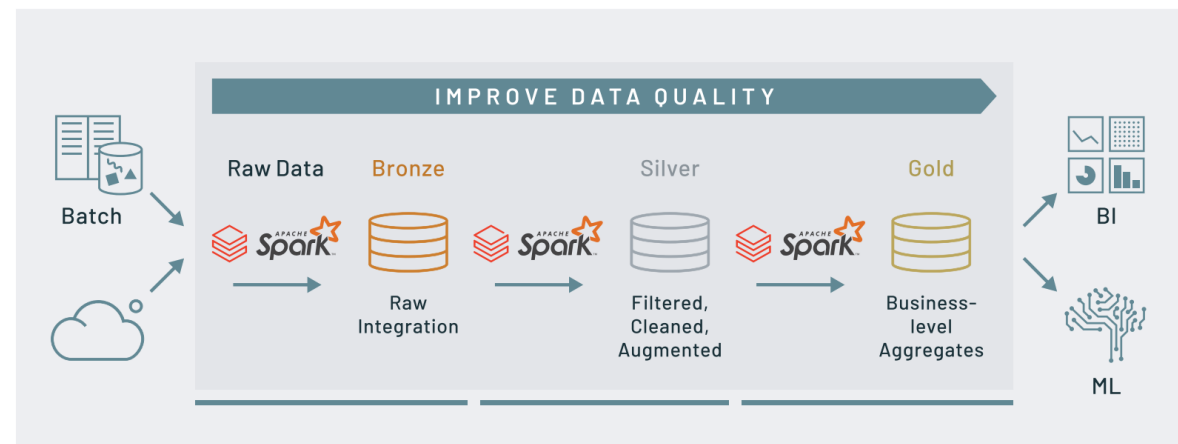


DATA LAKEHOUSE

- A lakehouse is a new architecture that combines the best elements of data lakes and data warehouses.
 - data warehouses → BI
 - data lakes → ML
- A lakehouse is a new DM architecture that enables users to do everything from BI, SQL analytics, data science, and ML on a single platform.
- A data lakehouse support
 - ACID transaction
 - Every operation is transactional
 - Real-time operations are consistent, and the historical data versions are automatically stored.
 - Large metadata (considered as data)
 - Indexing: lakehouse architecture includes various statistical techniques (bloom filters and data skipping) to avoid reading big portions of the data altogether
 - Schema validation: All your data that goes into a table must adhere strictly to a defined schema. If data doesn't satisfy the schema, it's moved into a quarantine where you can examine it later and resolve the issues.

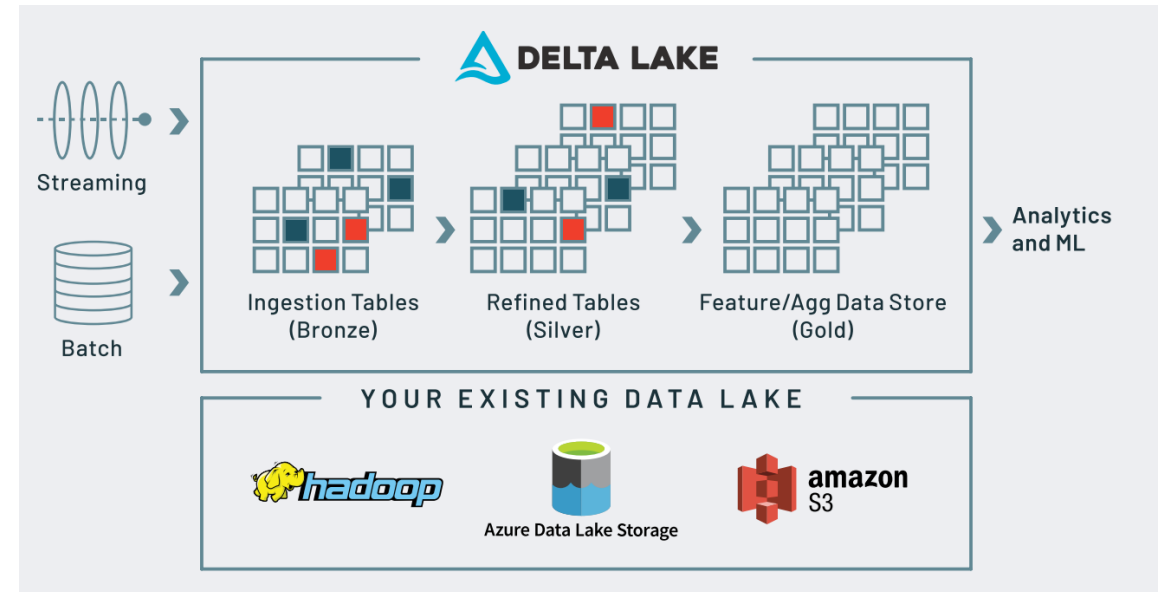
DELTA LAKE

- This process is powered by highly optimized Spark clusters and elastic cloud resources that can intelligently auto-scale up with increased workloads and auto-scale down for cost savings.
- **Delta Lake** is an open-source storage layer that brings data reliability to your existing data lake by providing
 - ACID, scalable metadata, unifying batch and streaming



DELTA LAKE

- Use of transaction log (single version of the truth)
 - ACID support via optimistic concurrency control
- Schema enforcement
 - Reject a write in a table if it does not match the schema
 - Schema validation on write
 - Table schema saved in JSON format in the log file
- Schema evolution
 - Read-compatible or non read compatible
- Time travel
 - Timestamp-based, version number, data retention
- Data lake is integrated with spark streaming



PROPERTIES OF BRONZE/SILVER/GOLD

- A medallion architecture is a data design pattern used to logically organize data in a lakehouse,
- incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture
- from Bronze $\Rightarrow$ Silver $\Rightarrow$ Gold layer tables

- Bronze tables
 - No data processing
 - Land all the data from external source systems.
 - The table structures correspond to the source system table structures "as-is,"
 - Data kept around for a couple weeks in order to fix mistakes just in case
- Silver tables
 - the data from the Bronze layer is matched, merged, conformed and cleansed
 - Directly queryable tables
- Gold tables
 - Data is typically organized in consumption-ready "project-specific" databases.
 - The Gold layer is for reporting and uses more de-normalized and read-optimized data models with fewer joins.
 - The final layer of data transformations and data quality rules are applied here.

